



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

آزمایش هشتم: ALU اعداد مختلط

نام درس

آزمایشگاه طراحی سیستم های دیجیتال

نیم سال ۱۴۰۴

استاد

دکتر انصاری

گروه ۸

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

حسن پناهی - ۴۰۳۱۰۵۸۳۳

امین استادکاظمی - ۴۰۳۱۷۰۸۹۳

فهرست مطالب

۱	مقدمه و هدف آزمایش	۲
۲	شرح آزمایش	۲
۳	ساختار مدار و بررسی کدهای پیاده‌سازی	۳
۳	۱.۳ ماژول Memory_32	۳
۴	۲.۳ ماژول Complex_ALU و طراحی FSM	۴
۷	۳.۳ ماژول Pipeline_CPU	۷
۴	تست مدار و waveform	۸
۸	۱.۴ کد وریلاگ تست بنچ	۸
۹	۲.۴ خروجی ها ، ویو فرم و تحلیل آن ها	۹
۵	نتیجه‌گیری	۱۰

۱ مقدمه و هدف آزمایش

هدف اصلی ما در این آزمایش، طراحی یک ALU برای انجام عملیات ریاضی (جمع، تفریق و ضرب) روی اعداد مختلط است. چالش اصلی در این آزمایش، محدودیت منابع سخت‌افزاری است؛ به این صورت که برای پیاده‌سازی کل این سیستم، فقط مجاز به استفاده از یک Adder و یک Multiplier هستیم.

با توجه به این محدودیت، نمی‌توانیم برای عملیاتی مثل ضرب اعداد مختلط سخت‌افزار جداگانه‌ای در نظر بگیریم. بنابراین رویکرد ما این است که با طراحی یک FSM، عملیات را به چند Clock Cycle بشکنیم تا بتوانیم از همان یک واحد جمع و ضرب به صورت مشترک استفاده کنیم.

در نهایت، این ALU را در کنار یک ماژول Memory قرار می‌دهیم تا یک Pipeline CPU شکل بگیرد. این ماشین پایه باید بتواند دستورات را از حافظه واکشی کرده و سناریوهای مختلف را اجرا کند.

۲ شرح آزمایش

در این آزمایش قصد داریم یک کامپیوتر پایه طراحی کنیم که وظیفه اصلی آن پردازش اعداد مختلط است. پیاده‌سازی به گونه‌ای انجام شده است که می‌تواند عملیات‌های ریاضی شامل جمع، تفریق و ضرب را روی دو عدد مختلط اعمال کند و دستورات مربوط به هر عملیات را به صورت متوالی از یک حافظه بخواند.

ورودی‌ها و خروجی‌های سیستم

در طراحی سطح بالای این سیستم، برای ارتباط با محیط بیرون و اعمال سناریوهای مختلف، پورت‌های زیر در نظر گرفته شده‌اند:

- **ورودی‌ها:** سیستم با clk کار می‌کند و یک ورودی rst برای ریست مدار دارد. داده‌های ورودی شامل دو عدد مختلط ۸ بیتی علامت‌دار هستند. این اعداد در قالب چهار ورودی مجزا برای بخش‌های حقیقی و موهومی دریافت می‌شوند.

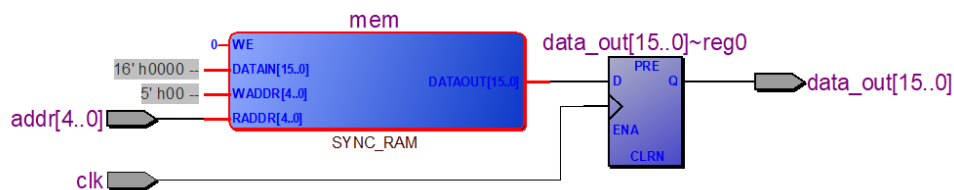
- **خروجی‌ها:** نتیجه نهایی محاسبات در قالب دو پورت ۱۶ بیتی (برای بخش حقیقی و موهومی) تولید می‌شود. برای اینکه مشخص شود عملیات ریاضی به اتمام رسیده و این خروجی‌ها معتبر هستند، یک سیگنال $done$ در نظر گرفته شده است که پس از پایان هر دستور، یک می‌شود. همچنین وضعیت PC را هم به عنوان خروجی قرار دادیم تا در مراحل تست، روند اجرای دستورات قابل رهگیری باشد.

۳ ساختار مدار و بررسی کدهای پیاده‌سازی

در این بخش، سیستم را به سه ماژول اصلی تقسیم کرده و ساختار هر کدام را بررسی می‌کنیم. در این پیاده‌سازی، هدف رعایت محدودیت منابع و برقراری ارتباط بین حافظه و واحد محاسباتی بوده است.

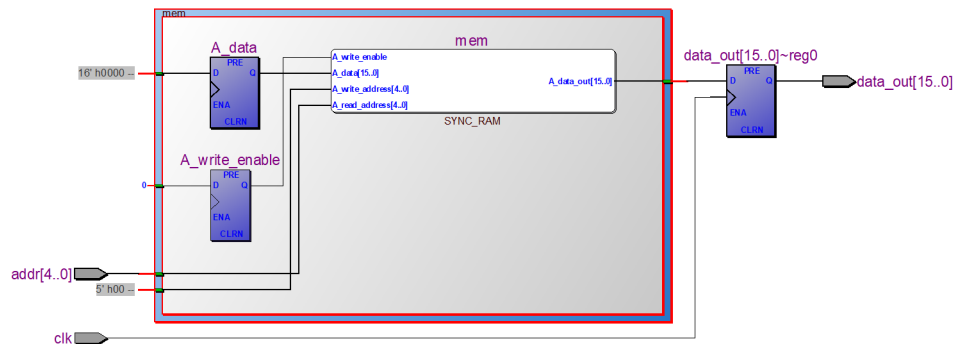
۱.۳ ماژول Memory_32

این ماژول همان حافظه دستورالعمل ماشین پایه است. ما یک آرایه ۳۲ خانه‌ای ۱۶ بیتی تعریف کردیم و ۶ خانه اول آن را برای تست مقداردهی کردیم:



شکل ۱: شماتیک RTL ماژول حافظه و استنتاج بلوک SYNC_RAM

در شماتیک بالا می‌بینیم که از بلوک‌های SYNC_RAM استفاده شده است. در نمای داخلی زیر هم مشخص است که خروجی با پالس 'clk' همگام است:



شکل ۲: نمای داخلی بلوک حافظه در Quartus

برای پیاده‌سازی این بخش، خواندن از حافظه را در یک بلوک 'always' حساس به لبه کلاک نوشتیم:

```
module Memory_32 (
    input clk,
    input [4:0] addr,
    output reg [15:0] data_out
);
    reg [15:0] mem [0:31];

    initial begin
        mem[0] = 16'h4000;
        mem[1] = 16'h8000;
        mem[2] = 16'hC000;
        mem[3] = 16'h4000;
        mem[4] = 16'hC000;
    end
```

```

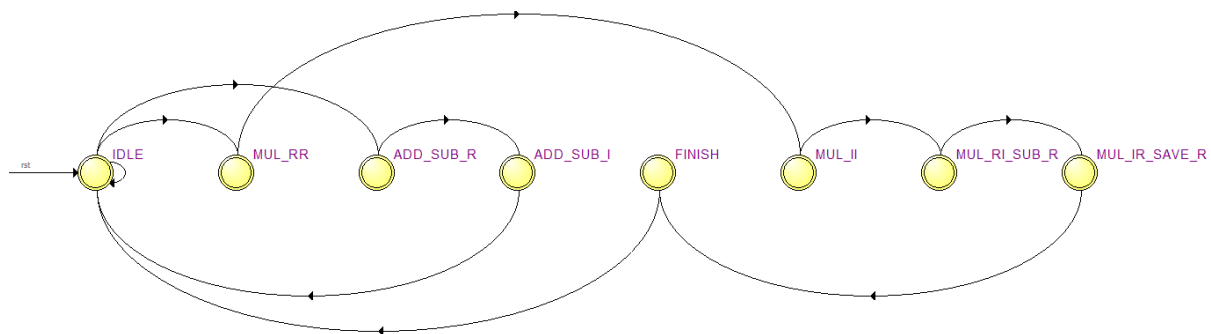
        mem[5] = 16'h0000;
    end

    always @(posedge clk) begin
        data_out <= mem[addr];
    end
endmodule

```

۲.۳ ماژول Complex_ALU و طراحی FSM

در این ماژول عملیات ریاضی روی اعداد مختلط انجام می‌شود. با توجه به محدودیت‌ها، فقط یک Adder و یک Multiplier تعریف کردیم. برای انجام عملیاتی مثل ضرب مختلط با همین منابع محدود، یک FSM با ۸ وضعیت طراحی کردیم که گراف آن را در تصویر زیر می‌بینید:



شکل ۳: دیاگرام گرافیکی State Machine استخراج شده از کد در Quartus

برای درک بهتر حالت‌ها، در اینجا مشخص شده است که در هر یک از این ۸ وضعیت دقیقاً چه فرآیندی طی می‌شود:

- **IDLE**: ماشین در این حالت منتظر می‌ماند. با یک شدن سیگنال start، بر اساس مقدار op تصمیم می‌گیرد که وارد مسیر جمع/تفریق شود یا مسیر ضرب.
- **ADD_SUB_R**: در این مرحله، بخش حقیقی دو عدد (Ar و Br) به Adder فرستاده شده و نتیجه در خروجی Cr ذخیره می‌شود.
- **ADD_SUB_I**: بخش موهومی دو عدد (Ai و Bi) به Adder داده می‌شود. نتیجه در Ci ذخیره شده و با یک شدن سیگنال ready، ماشین به IDLE برمی‌گردد.
- **MUL_RR**: این اولین مرحله از مسیر ضرب است. Ar و Br به ضرب‌کننده داده شده و نتیجه موقتاً در رجیستر temp1 ذخیره می‌شود.
- **MUL_II**: مقادیر Ai و Bi در هم ضرب شده و در رجیستر temp2 ریخته می‌شوند.
- **MUL_RI_SUB_R**: در این حالت دو کار همزمان انجام می‌شود: اول اینکه temp1 و temp2 (که نتایج ضرب‌های قبلی هستند) به Adder داده می‌شوند تا از هم کم شوند و بخش حقیقی جواب یعنی Cr به دست بیاید. همزمان، ضرب ضربدری Bi و Ar انجام شده و مقدار جدید در temp1 ذخیره می‌شود.

● **MUL_IR_SAVE_R**: ضرب ضربداری دوم یعنی Ai در Br انجام شده و نتیجه در temp2 ذخیره می‌شود.

● **FINISH**: در مرحله آخر، مقادیر جدید temp1 و temp2 با هم جمع می‌شوند تا بخش موهومی جواب ساخته شود و در Ci قرار بگیرد. سیگنال ready یک شده و عملیات تمام می‌شود.

در جدول زیر، شروط انتقال بین این حالت‌ها مشخص شده است:

	Source State	Destination State	Condition
1	ADD_SUB_I	IDLE	
2	ADD_SUB_R	ADD_IDLE	
3	FINISH	IDLE	
4	IDLE	MUL_RR	(op[0]).(op[1]).(!always0).(start)
5	IDLE	IDLE	(!op[0]).(!always0) + (!op[0]).(always0).(start) + (op[0]).(!op[1]).(!always0) + (op[0]).(!op[1]).(always0).(start) + (op[0]).(op[1]).(!start)
6	IDLE	ADD_SUB_R	(always0).(start)
7	MUL_II	MUL_RI_SUB_R	
8	MUL_IR_SAVE_R	FINISH	
9	MUL_RI_SUB_R	MUL_IR_SAVE_R	
10	MUL_RR	MUL_II	

شکل ۴: جدول انتقال حالت‌ها (State Transition Table)

در کد زیر، FSM با استفاده از یک بلوک ترتیبی برای تغییر وضعیت و یک بلوک ترکیبی برای اعمال متغیرها به مدار محاسباتی پیاده‌سازی شده است:

```
module Complex_ALU (
    input clk,
    input rst,
    input start,
    input [1:0] op,
    input signed [7:0] Ar, Ai, Br, Bi,
    output reg signed [15:0] Cr, Ci,
    output reg ready
);

    localparam IDLE = 3'd0;
    localparam ADD_SUB_R = 3'd1;
    localparam ADD_SUB_I = 3'd2;
    localparam MUL_RR = 3'd3;
    localparam MUL_II = 3'd4;
    localparam MUL_RI_SUB_R = 3'd5;
    localparam MUL_IR_SAVE_R = 3'd6;
    localparam FINISH = 3'd7;

    reg signed [15:0] add_in1, add_in2;
    reg add_sub;
    wire signed [15:0] add_out;
    assign add_out = add_sub ? (add_in1 - add_in2) : (add_in1 + add_in2);

    reg signed [7:0] mul_in1, mul_in2;
    wire signed [15:0] mul_out;
    assign mul_out = mul_in1 * mul_in2;

    reg [2:0] state;
    reg signed [15:0] temp1, temp2;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            state <= IDLE;
            ready <= 0;
            Cr <= 0;
            Ci <= 0;
        end else begin
            case (state)
                IDLE: begin
                    ready <= 0;
                    if (start) begin
                        if (op == 1 || op == 2) state <= ADD_SUB_R;
                        else if (op == 3) state <= MUL_RR;
                    end
                end
                ADD_SUB_R: begin
                    Cr <= add_out;
                end
            endcase
        end
    end
```

```

        state <= ADD_SUB_I;
    end
    ADD_SUB_I: begin
        Ci <= add_out;
        ready <= 1;
        state <= IDLE;
    end
    MUL_RR: begin
        temp1 <= mul_out;
        state <= MUL_II;
    end
    MUL_II: begin
        temp2 <= mul_out;
        state <= MUL_RI_SUB_R;
    end
    MUL_RI_SUB_R: begin
        Cr <= add_out;
        temp1 <= mul_out;
        state <= MUL_IR_SAVE_R;
    end
    MUL_IR_SAVE_R: begin
        temp2 <= mul_out;
        state <= FINISH;
    end
    FINISH: begin
        Ci <= add_out;
        ready <= 1;
        state <= IDLE;
    end
    default: state <= IDLE;
endcase
end
end

always @(*) begin
    add_in1 = 0; add_in2 = 0; add_sub = 0;
    mul_in1 = 0; mul_in2 = 0;

    case (state)
        ADD_SUB_R: begin
            add_in1 = {{8{Ar[7]}}, Ar};
            add_in2 = {{8{Br[7]}}, Br};
            if (op == 2) add_sub = 1;
            else add_sub = 0;
        end
        ADD_SUB_I: begin
            add_in1 = {{8{Ai[7]}}, Ai};
            add_in2 = {{8{Bi[7]}}, Bi};
            if (op == 2) add_sub = 1;
            else add_sub = 0;
        end
        MUL_RR: begin
            mul_in1 = Ar; mul_in2 = Br;
        end
        MUL_II: begin
            mul_in1 = Ai; mul_in2 = Bi;
        end
        MUL_RI_SUB_R: begin
            add_in1 = temp1; add_in2 = temp2; add_sub = 1;
            mul_in1 = Ar; mul_in2 = Bi;
        end
        MUL_IR_SAVE_R: begin
            mul_in1 = Ai; mul_in2 = Br;
        end
        FINISH: begin
            add_in1 = temp1; add_in2 = temp2; add_sub = 0;
        end
    endcase
end
endmodule

```

۳.۳ ماژول Pipeline_CPU

این ماژول در سطح Top-Level قرار دارد و از طریق سیمکشی، ماژول‌های حافظه و ALU را به هم متصل می‌کند.

روند کار به این صورت است که متغیر PC به عنوان آدرس به حافظه داده می‌شود. چون عملیات‌های ALU چند سیکل طول می‌کشد، این ماژول طوری نوشته شده که تا وقتی سیگنال ready یک نشده، PC متوقف بماند. در کد زیر، برای هماهنگی داده‌ها و جلوگیری از تداخل ورودی‌ها در حین پردازش، از رجیسترهای میانی استفاده شده است.

```
module Pipeline_CPU (
    input clk,
    input rst,
    input signed [7:0] data_Ar, data_Ai, data_Br, data_Bi,
    output signed [15:0] result_r,
    output signed [15:0] result_i,
    output done,
    output [4:0] current_pc
);
    reg [4:0] pc;
    wire [15:0] instr;
    reg [15:0] if_id_instr;
    reg id_ex_start;
    reg [1:0] id_ex_op;
    reg signed [7:0] id_ex_Ar, id_ex_Ai, id_ex_Br, id_ex_Bi;
    wire alu_ready;
    wire [15:0] alu_Cr, alu_Ci;

    Memory_32 u_mem (
        .clk(clk),
        .addr(pc),
        .data_out(instr)
    );

    Complex_ALU u_alu (
        .clk(clk),
        .rst(rst),
        .start(id_ex_start),
        .op(id_ex_op),
        .Ar(id_ex_Ar),
        .Ai(id_ex_Ai),
        .Br(id_ex_Br),
        .Bi(id_ex_Bi),
        .Cr(alu_Cr),
        .Ci(alu_Ci),
        .ready(alu_ready)
    );

    assign result_r = alu_Cr;
    assign result_i = alu_Ci;
    assign done = alu_ready;
    assign current_pc = pc;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            pc <= 0;
            if_id_instr <= 0;
            id_ex_start <= 0;
        end else begin
            if (alu_ready || pc == 0) begin
                if_id_instr <= instr;
                id_ex_op <= instr[15:14];

                if (instr[15:14] != 0) begin
                    pc <= pc + 1;
                    id_ex_start <= 1;
                    id_ex_Ar <= data_Ar;
                    id_ex_Ai <= data_Ai;
                    id_ex_Br <= data_Br;
                    id_ex_Bi <= data_Bi;
                end else begin
                    id_ex_start <= 0;
                end
            end else begin
                id_ex_start <= 0;
            end
        end
    end
endmodule
```


۴ تست مدار و waveform

۱.۴ کد وریلاگ تست بنچ

```
`timescale 1ns / 1ps

module tb_Pipeline_CPU();

    reg clk;
    reg rst;
    reg signed [7:0] test_Ar, test_Ai, test_Br, test_Bi;
    wire signed [15:0] result_r;
    wire signed [15:0] result_i;
    wire done;
    wire [4:0] current_pc;

    Pipeline_CPU uut (
        .clk(clk),
        .rst(rst),
        .data_Ar(test_Ar),
        .data_Ai(test_Ai),
        .data_Br(test_Br),
        .data_Bi(test_Bi),
        .result_r(result_r),
        .result_i(result_i),
        .done(done),
        .current_pc(current_pc)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    always @(*) begin
        case (current_pc)
            5'd0: begin test_Ar = 3; test_Ai = 4; test_Br = 2; test_Bi = 1; end
            5'd1: begin test_Ar = -5; test_Ai = 2; test_Br = 1; test_Bi = -3; end
            5'd2: begin test_Ar = 2; test_Ai = 3; test_Br = 4; test_Bi = 5; end
            5'd3: begin test_Ar = 0; test_Ai = 0; test_Br = -7; test_Bi = 8; end
            5'd4: begin test_Ar = -2; test_Ai = -1; test_Br = -3; test_Bi = -2; end
            default: begin test_Ar = 0; test_Ai = 0; test_Br = 0; test_Bi = 0; end
        endcase
    end

    integer op_count = 0;

    initial begin
        $display("Starting Simulation...");
        rst = 1;
        #15;
        rst = 0;

        wait(op_count == 5);
        #20;
        $display("Simulation done.");
        $stop;
    end

    always @(posedge clk) begin
        if (done) begin
            op_count = op_count + 1;

            case (op_count)
                1: begin
                    $display("Test1(ADD) In: A=(3+4i), B=(2+1i) Res=%0d+%0di Exp: 5+5i", result_r, result_i);
                    if (result_r == 5 && result_i == 5) $display("U->PASS"); else $display("U->FAIL");
                end
                2: begin
                    $display("Test2(SUB) In: A=(-5+2i), B=(1-3i) Res=%0d+%0di Exp: -6+5i", result_r, result_i);
                    if (result_r == -6 && result_i == 5) $display("U->PASS"); else $display("U->FAIL");
                end
                3: begin
                    $display("Test3(MUL) In: A=(2+3i), B=(4+5i) Res=%0d+%0di Exp: -7+22i", result_r, result_i);
                    if (result_r == -7 && result_i == 22) $display("U->PASS"); else $display("U->FAIL");
                end
                4: begin
                    $display("Test4(ADD) In: A=(0+0i), B=(-7+8i) Res=%0d+%0di Exp: -7+8i", result_r, result_i);
                    if (result_r == -7 && result_i == 8) $display("U->PASS"); else $display("U->FAIL");
                end
                5: begin
                    $display("Test5(MUL) In: A=(-2-1i), B=(-3-2i) Res=%0d+%0di Exp: 4+7i", result_r, result_i);
                    if (result_r == 4 && result_i == 7) $display("U->PASS"); else $display("U->FAIL");
                end
            endcase
        end
    end

endmodule
```

۲.۴ خروجی ها ، ویو فرم و تحلیل آن ها

پس از اجرای تست بنچ در نرم افزار ModelSim، شکل موج اجرای متوالی ۵ دستور از حافظه به دست آمد. همچنین خروجی پیام های چاپ شده در ترمینال شبیه ساز، وضعیت هر تست را گزارش می دهد.

```
# Starting Simulation...
# Test 1 (ADD) | In: A=(3+4i), B=(2+1i) | Res = 5+5i | Exp: 5+5i
# -> PASS
# Test 2 (SUB) | In: A=(-5+2i), B=(1-3i) | Res = -6+5i | Exp: -6+5i
# -> PASS
# Test 3 (MUL) | In: A=(2+3i), B=(4+5i) | Res = -7+22i | Exp: -7+22i
# -> PASS
# Test 4 (ADD) | In: A=(0+0i), B=(-7+8i) | Res = -7+8i | Exp: -7+8i
# -> PASS
# Test 5 (MUL) | In: A=(-2-1i), B=(-3-2i) | Res = 4+7i | Exp: 4+7i
# -> PASS
# Simulation done.
```

شکل ۵: لاگ های چاپ شده در ترمینال شبیه ساز

با بررسی دقیق شکل موج در بازه های زمانی مختلف (که با متغیر op_count مشخص شده اند)، رفتار مدار در هر سناریو به شرح زیر تحلیل می شود:

- **تست ۱ (ADD):** در این بازه، current_pc روی مقدار ۰۰۰۰۰ قرار دارد و دستور جمع از حافظه خوانده شده است. ورودی های مدار $(3+4i)$ و $(2+1i)$ هستند. مدار در حدود ۳ سیکل کلاک این مقادیر را پردازش می کند. در لبه بالا رونده پالس done، مقادیر خروجی های result_r و result_i روی اعداد ۵ و ۵ تثبیت می شوند.

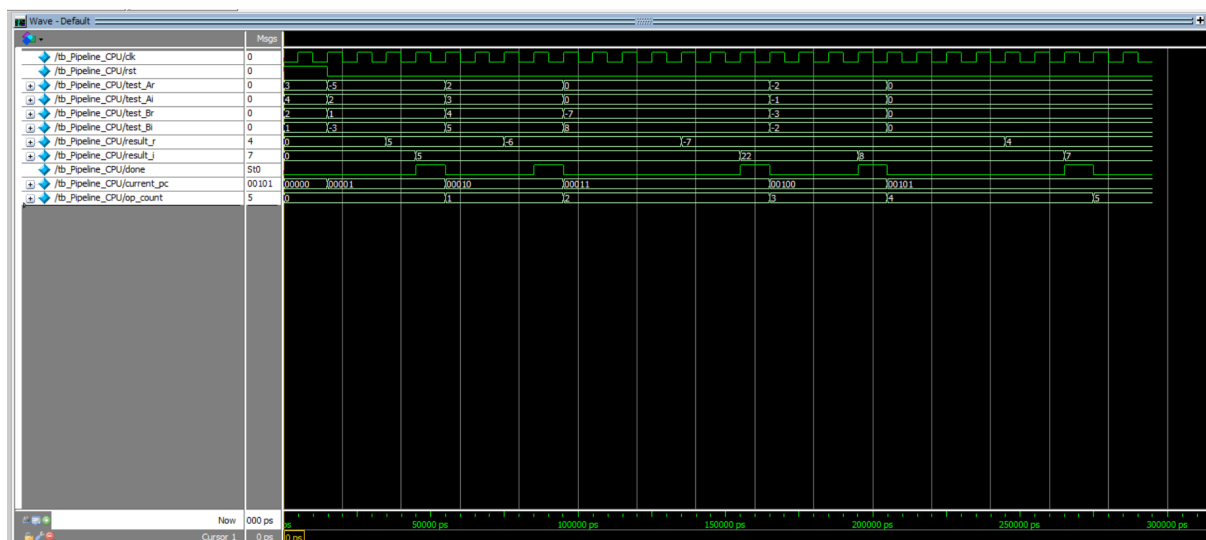
- **تست ۲ (SUB):** شمارنده برنامه یک واحد اضافه شده (۰۰۰۰۱) . در این سناریو، تفریق اعداد دارای علامت تست شده است. ورودی ها $(-5+2i)$ و $(1-3i)$ هستند. رجیسترهای خروجی در انتهای پردازش مقادیر ۶- و ۵ را در خود نگه می دارند که نشان می دهد واحد Adder مدار، عملیات تفریق با در نظر گرفتن بیت علامت را به درستی اجرا کرده است.

- **تست ۳ (MUL):** در سیکل بعدی، دستور ضرب واکشی می شود. ورودی ها اعداد مثبت $(2+3i)$ و $(4+5i)$ هستند. نکته مهم در شکل موج این است که فاصله زمانی بین شروع این دستور تا فعال شدن سیگنال done، تقریباً دو برابر حالت های قبلی (حدود ۶ سیکل کلاک) است. این تاخیر نشان می دهد که ماشین حالت در حال استفاده اشتراکی از تنها یک بلوک Multiplier در مدار است. در نهایت مقادیر ۷- و ۲۲ در خروجی قرار می گیرند.

- **تست ۴ (حالت مرزی ADD):** در این تست، جمع با عدد صفر بررسی شده است. ورودی ها $(0+0i)$ و $(-7+8i)$ هستند. همان طور که در شکل موج مشخص است، خروجی های result بدون

تغییر نسبت به ورودی دوم، روی همان ۷- و ۸ تنظیم شده و پالس done به سرعت یک می شود.

- تست ۵ (MUL با اعداد منفی): آخرین سناریو، بررسی ضرب دو عدد منفی است تا منطق علامت ارزیابی شود. ورودی ها $(-2-1i)$ و $(-3-2i)$ هستند. ماشین حالت مانند تست سوم، سیکل های طولانی خود را طی می کند و در نهایت نتیجه $(4+7i)$ در خروجی ظاهر می شود.



شکل ۶: شکل موج سیگنال های ورودی، خروجی و کنترلی مدار در اجرای ۵ سناریو

۵ نتیجه گیری

در این آزمایش، یک ALU برای پردازش اعداد مختلط طراحی کردیم و آن را به همراه حافظه در قالب یک ماشین پایه پیاده سازی کردیم. چالش اصلی کار، محدودیت استفاده از تنها یک Adder و یک Multiplier بود. برای رفع این محدودیت، از منطق FSM کمک گرفتیم تا عملیات های سنگینی مثل ضرب مختلط را در چند Clock Cycle بشکنیم و از منابع سخت افزاری به صورت مشترک استفاده کنیم. نتایج شبیه سازی در ModelSim نیز نشان داد که مدار در تمامی سناریوهای تست (اعداد مثبت، منفی و صفر) به درستی کار می کند.